

Python ML by Example

AI Stuart Russel (Pearson)

→ AI: Mimic Human Intelligence

↳ Making machines intelligent and enable to take action & decisions that humans do

→ Machine - Makes life easier

↳ Uses power to control mechanical movements

Complex machines: 2 or more simple machines

→ Intelligence: Ability to understand, learn & think

→ 2 dimensions of AI:

(1) Humans vs. Rational

(2) Act vs. Think

→ Acting Humanly Turing Test

Total Turing Test

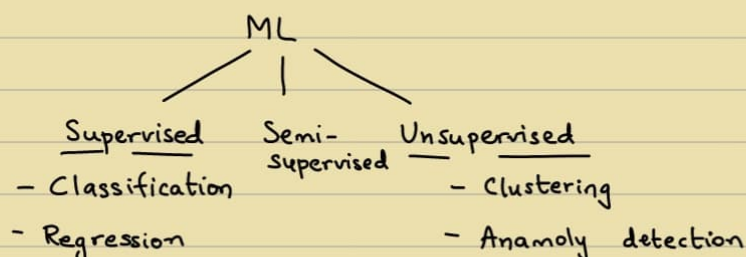
→ (1) Think Humanly

(2) Act Rationally

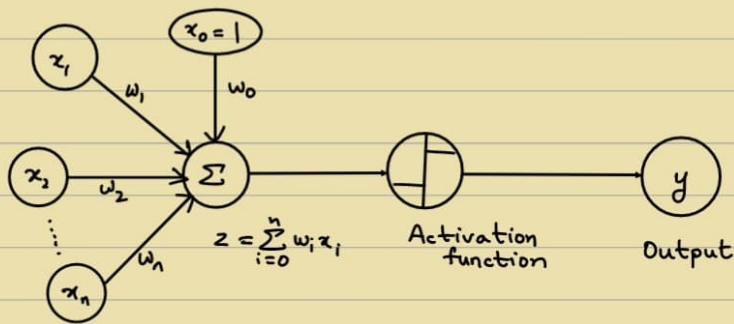
(3) Think Rationally

→ ML: Construction of Algos that can learn from & make predictions on data.

↳ features



→ Perceptron Model



$$y = f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

→ Knowledge Base :

Set of Sentences represented in knowledge representation language.

Expressing knowledge explicitly in a computer tractable way
(logic)

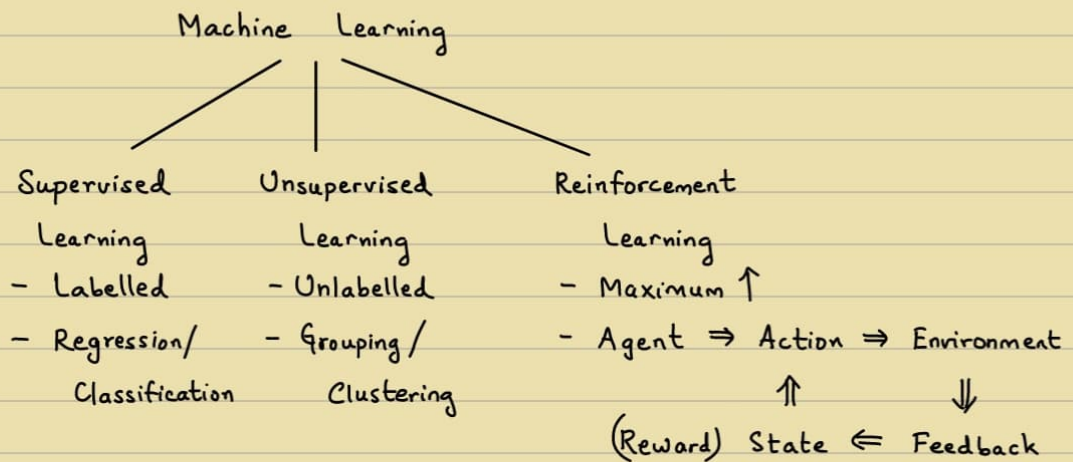
→ Logic : A way of thinking or explaining something

↳ Basis of all mathematical & Automated reasoning

17/1

→ Machine Learning :

- Perform specific tasks effectively without explicit instructions
- Relies on Patterns and Inference instead.
- Involves building a math model based on training data (sample data) to make predictions / decisions
- Why ML :
 - Data ↑ - Various formats
 - Patterns ↑



→ AlphaGo :

MuZero → Without Rules

21/1

Machine Learning

- Term coined around 1960
- Why learn ? Why not just hire enough programmers and code in rules ?
 - Lots of patterns for an activity/event
 - Events can be dynamic
 - Data is increasing exponentially
 - Data is also in various formats [Text, Audio, Video]
 - Higher quality data due to cheaper storage
- Can be broadly classified into three categories
 - Unsupervised, Supervised and Reinforcement learning

AlphaGo

- First computer Go program to defeat a 9-dan professional player
- Uses Monte Carlo Tree search algorithm based on knowledge learned by a deep learning method
- Beat World No. 1 ranked player in 2017
 - Retired after this match
- <https://deepmind.google/research/breakthroughs/alphago/>
- <https://www.youtube.com/watch?v=WXuK6gekU1Y>
- AlphaGo Zero – Version without human data and stronger than AlphaGo [defeated 100-0]

↪ Reinforcement learning
(Design incentive structure)

• Search:

Initially AI used BFS, then later switched to DFS.

- (1) BFS
 - (2) DFS
 - (3) A*
- } - Non-Randomised

(4) Monte Carlo Tree Search - Randomized Search Technique

Simulation - performed by Neural Networks (AlphaGo)

→ Which direction to go to ?

AlphaZero & MuZero

- AlphaZero, a generalized version of AlphaGo Zero Took 4 hours to learn Chess and defeat reigning world computer chess champion 28 to 0 in 100 matches
- https://www.youtube.com/watch?time_continue=7&v=tXIM99xPQC8
- MuZero: Master games without knowing rules
- Uses approach similar to AlphaZero, developed in 2019

AlphaFold: solution to a 50 year old grand challenge in biology

- <https://deepmind.com/blog/article/alphafold-a-solution-to-a-50-year-old-grand-challenge-in-biology>
- Figuring out what shapes proteins fold into is known as the "protein folding problem" - grand challenge in biology for the past 50 years
- Focus of intensive scientific research for many years, using a variety of experimental techniques such as nuclear magnetic resonance and X-ray crystallography.

AlphaFold

- Number of ways a protein could theoretically fold before settling into its final 3D structure is astronomical.
- Cyrus Levinthal estimated 10^{300} possible conformations for a typical protein.
- Estimated would take longer than the age of universe to enumerate all possible configurations. Yet in nature, proteins fold spontaneously, some within milliseconds - referred to as Levinthal's paradox.

Nobel Prize in Chemistry 2024 !!!

- The Nobel Prize in Chemistry 2024 is about proteins, life's ingenious chemical tools.
- David Baker has succeeded with the almost impossible feat of building entirely new kinds of proteins.
- Demis Hassabis and John Jumper have developed an AI model to solve a 50-year-old problem: predicting proteins' complex structures. These discoveries hold enormous potential.

AI Index 2025

- Source: Stanford Institute for Human-Centered Artificial Intelligence (HAI)
- Report: 2025 AI Index Report
- <https://hai.stanford.edu/ai-index/2025-ai-index-report>
- Slides follow

AI Performance on Demanding Benchmarks

- AI achieved large gains on harder benchmarks
- Strong progress in coding and video generation
- AI agents outperform humans in time-constrained tasks

Industry Dominates AI Development

- ~90% of notable models built by industry
- Performance gaps narrowed
- Compute scale continues to rise - training compute doubles every five months, datasets every eight, and power use annually.

→ Cross - Validation :

Partitioning of training dataset.

k parts $\equiv$ Training : $k-1$ parts
 Testing : k^{th} part } → Repeat 'k' times
 k ↓

- Holdout Method - Non-exhaustive Cross Validation

Con : Some data never gets into the test data



- Exhaustive Cross-Validation :

LOOC - Used for Sparse dataset

- Nested Cross Validation :

k -fold Cross validation with validation and Test

Perform K -fold on the partitioned data (without Test)

→ Regularization :

Penalty function

Reduce importance for some terms

Occam's Razor : Simplest hypothesis that fits data should be preferred. (Not Complex model)

(i) Early Stopping

Given :

$$f(x) = \theta + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

$$MSE = \frac{1}{m} \cdot \min_{\theta, \theta_1, \theta_2, \theta_3} \sum_{i=1}^m (\hat{f}_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\text{Loss} : J(\theta)$$

Adding Penalty : $1000 \theta_3 + 1000 \theta_4$

(ii) Feature Selection

Pick top k Heuristically useful features

n features : 2^n feature combinations

Filter Methods : Correlation (Chi-Squared for Categorical)
 Pearson coeff. for Continuous

11/2

→ Feature Selection : (Reduce overfitting)

2^n total

(i) Filter Methods :

x , $2x$ - High correlation, $\therefore$ Remove Redundancy

(ii) Wrapped methods :

Underfit Model Search

Recursive feature elimination : Remove best feature and construct the next model iteratively (Worst)

(iii) Embedded methods :

Assess features acc. to Accuracy

e.g. Regularisation - Add Penalty in loss function

• Dimensionality Reduction :

- Information loss : Minimum

• Data Preprocessing :

(Data Mining)

- CRISP DM - Standard

- Data preparation $\Rightarrow$ Modeling $\Rightarrow$ Evaluation $\Rightarrow$ Deployment

- Data Cleaning - ★

Domain knowledge - ★

- Manual work (Numerical)

- Features can be quantitative / Categorical.

- Feature Engg. : Improving features

- Missing values

→ Ignore

→ Substitute with fixed values : Imputing

- Label Encoder : Label $\Rightarrow$ Numeric value

(Integer Label Encoder) - ML infers ordering based on encoding.

Done by One-hot encoding

- Scaling :

(i) Standardization : $\mu = 0$ & $\sigma^2 = 1$ $\left(\frac{z - \mu}{\sigma} \right)$

If not Gaussian, Subtract Median & Divide by Interquartile
1st (25%) and 3rd (75%) Range

(ii) Normalization : min(0) - max(1) Scaling

- Polynomial features :

Occam's Razor : Avoid Higher order & interaction terms
 $a^2 + ab + b^2$, ab : interaction term

$$y = w_1x_1 + w_2x_2 + w_3$$

- Power transformations :

- Box-Cox transform : Find best power
- Binning : Separation of feature values into bins
(Lossy transformation)

→ Uncertainty - Decision Theory

Uncertainty

- May not have categorical answers
 - May not know exactly which route is slow
 - 50% chance that one route is slow
 - Even if we know it is slow, how slow?
- Uncertainty changes the way an agent makes a decision
- Probability theory used in helping agents reason with uncertainty
 - Summarize uncertainty due to our ignorance or laziness
 - May be very very costly to remove such uncertainty
- Rational decision depends on
 - Relative importance of various goals
 - Likelihood or degree to which they will be achieved

Decisions under Uncertainty

- How should an agent make a decision:
 - Plan 1, 80% chance it will succeed (Route 1)
 - Route 1 will be quick if flowing smoothly, but stuck for 1 hour if accident
 - If plan1 is successful the outcome is very good, but failure is very bad
 - Plan 2, 70% chance it will succeed (Route 2)
 - Route 2 will be somewhat quick, but not so bad even if slow
 - If plan2 successful the outcome is good (not as good as plan1's success), but failure is not that bad

Basic Probability Notation

- Suppose we have probabilities over all possible values of a variable -- denote by $P(X)$
 - Called the probability distribution for the random variable:
 - $P(\text{Weather} = \text{sunny}) = 0.6$
 - $P(\text{Weather} = \text{rain}) = 0.1$
 - $P(\text{Weather} = \text{cloudy}) = 0.29$
 - $P(\text{Weather} = \text{snow}) = .01$
 - $P(\text{Weather}) = \langle 0.6, 0.1, 0.29, 0.01 \rangle$
- $$\sum_{x_i \in \{\text{sunny, rain, cloudy, snow}\}} P(X=x_i) = 1$$

Diagnosis domain: Introducing Cavity Variable

- Doctor diagnosing a dental patient's toothache
- Propositional logic rules
- Toothache $\rightarrow$ Cavity [Toothache implies Cavity] (Incomplete rule)
- Toothache $\rightarrow$ Cavity $\vee$ Gum Problem $\vee$ Abscess ... (Lots of such problems)
- Turning the rule around, Cavity $\rightarrow$ Toothache
 - Not totally right either, need to make it logically exhaustive

Inference using full joint distribution

	toothache		$\neg$ toothache	
	catch	$\neg$ catch	catch	$\neg$ catch
cavity	.108	.012	.072	.008
$\neg$ cavity	.016	.064	.144	.576

- 3 variables: {Cavity, Toothache, Catch}
- Dentist probe catches or not
- 2 * 2 * 2 table
- $P(\text{Cavity}) = .108 + .012 + .072 + .008 = 0.2$ (Called **Marginal Probability** of Cavity)

Marginalization (or summing out)

- For the variable set Z not including variable Y

$$P(Y) = \sum_{z \in Z} P(Y, z) = \sum_z P(Y | z) P(z)$$

$$P(\text{Cavity}) = \sum_{z \in \{\text{Catch, Toothache}\}} P(\text{Cavity}, z)$$
- The above rule is called conditioning (conditioned on z)
- Marginalization** and **conditioning** very useful rules

Marginalization

$$P(\text{cavity} | \text{toothache}) = \frac{P(\text{cavity} \wedge \text{toothache})}{P(\text{toothache})}$$

$$= \frac{0.108 + 0.012}{(0.108 + 0.012 + 0.016 + 0.064)} = 0.6$$

$$P(\neg \text{cavity} | \text{toothache}) = \frac{P(\neg \text{cavity} \wedge \text{toothache})}{P(\text{toothache})}$$

$$= \frac{0.016 + 0.064}{.108 + .012 + .016 + .064} = .4$$

$$P(\text{cavity} | \text{toothache}) + P(\neg \text{cavity} | \text{toothache}) = 1$$

Marginalization

- Denominator can be viewed as a normalization constant $1/P(\text{toothache})$
- $P(\text{Cavity} | \text{Toothache}) = \alpha P(\text{Cavity, toothache}) = \alpha [P(\text{Cavity, toothache, catch}) + P(\text{Cavity, toothache, } \neg \text{catch})] = .108 + .012 = \alpha 0.12$
- $\alpha P(\neg \text{Cavity, toothache}) = \alpha [.016 + .064] = \alpha 0.08$
- Normalizing $\alpha \langle .12, .08 \rangle$ gives $\langle .6, .4 \rangle$
- Issue:** For a domain with n Boolean variables, input table size of $O(2^n)$
- Full joint distribution in tabular form not a practical tool for building reasoning systems

Marginalization

- Denominator can be viewed as a normalization constant $1/P(\text{toothache})$
- $P(\text{Cavity} | \text{Toothache}) = \alpha P(\text{Cavity}, \text{toothache}) = \alpha [P(\text{Cavity}, \text{toothache}, \text{catch}) + P(\text{Cavity}, \text{toothache}, \neg \text{catch})] = .108 + .012 = \alpha 0.12$
- $\alpha P(\neg \text{Cavity}, \text{toothache}) = \alpha [.016 + .064] = \alpha 0.08$
- Normalizing $\alpha <.12, .08>$ gives $<.6, .4>$
- **Issue:** For a domain with n Boolean variables, input table size of $O(2^n)$
- Full joint distribution in tabular form not a practical tool for building reasoning systems

Independence

- Helps in reducing size of domain representation and complexity of inference problem
- Let's add weather into the example : sunny, rain, cloudy, snow $\rightarrow 2 * 2 * 2 * 4$ entries in table
- $P(\text{toothache}, \text{catch}, \text{cavity}, \text{cloudy}) = P(\text{cloudy} | \text{toothache}, \text{catch}, \text{cavity}) P(\text{toothache}, \text{catch}, \text{cavity})$
- $P(\text{cloudy} | \text{toothache}, \text{catch}, \text{cavity}) = P(\text{cloudy})$

Bayes' Rule

- Given that
 - $P(A \wedge B) = P(A|B) * P(B)$
 - $P(A \wedge B) = P(B|A) * P(A)$
 - $P(B|A) = (P(A|B) * P(B)) / P(A)$
- We can determine $P(B|A)$ given $P(A|B)$, $P(B)$ and $P(A)$
- $P(\text{effect} | \text{cause})$ may be **causal knowledge**, $P(\text{cause} | \text{effect})$ **diagnostic knowledge**
- Often have conditional probabilities on causal relationships $\rightarrow P(\text{symptoms} | \text{disease})$
 - Need to infer diagnostic knowledge many times

Bayes' Rule Example

- S: Proposition that patient has stiff neck
- M: Proposition that patient has meningitis
- Meningitis causes stiff-neck, 70% of the time
- Given:
 - $P(S | M) = 0.7$
 - $P(M) = 1/50,000$
 - $P(S) = .01$
 - $P(M|S) = P(S | M) * P(M) / P(S) = (.7 * 1/50000) / .01 = .0014$

Bayes' Rule Use

- Why not estimate $P(M | S)$ the same way as we do $P(S | M)$?
 - Why do we need Bayes' rule?
- $P(S | M)$ is **causal knowledge**, does not change
 - It is "model based"
 - It reflects the way meningitis works
- $P(M | S)$ is **diagnostic**; tells us likelihood of M given symptom S
 - Diagnostic knowledge may change with circumstance, thus helpful to derive it
 - If there is an epidemic, probability of Meningitis goes up; rather than trying to estimate $P(M | S)$ which is very hard, we can compute it using the other terms
 - However, $P(S | M)$ is unaffected by the epidemic

Computing the denominator: $P(S)$

We wish to avoid computing the denominator in the Bayes' rule

- May be hard to obtain
- In our example, the denominator is $P(S)$
 - Techniques to compute (or avoid computing $P(S)$)

Our first approach is to compute relative likelihoods

- If M (meningitis) and W (whiplash) are two possible explanations:
 - $P(M|S) = P(S | M) * P(M) / P(S)$
 - $P(W|S) = P(S | W) * P(W) / P(S)$
- $P(M|S) / P(W|S) = P(S | M) * P(M) / P(S | W) * P(W)$

More General Forms of Bayes' Rule

- $P(Y | X) = P(X | Y) * P(Y) / P(X)$
- Bayes' rule for multi-valued variables
- Generalize to some background evidence e
- $P(Y | X, e) = P(X | Y, e) * P(Y | e) / P(X | e)$

Conditional Independence

- Toothache and Catch are independent given the presence of Cavity
 - Each is directly caused by cavity, toothache is caused by cavity and a dentist catches cavity using probe
 - A toothache cannot be caught by a probe nor a probe results in a toothache
- Conditional independence captured as : $P(\text{toothache} \wedge \text{catch} | \text{cavity}) = P(\text{toothache} | \text{catch}, \text{cavity}) P(\text{catch} | \text{cavity})$
- $P(\text{toothache} | \text{catch}, \text{cavity}) = P(A | B, C)$
 - If A and C are conditionally independent given B
 - Then, probability of A is not dependent on C
 - i.e. $P(A | B \wedge C) = P(A | B)$
- Therefore $P(\text{toothache}, \text{catch} | \text{cavity}) = P(\text{toothache} | \text{cavity}) P(\text{catch} | \text{cavity})$

Combining Evidence

- Revisiting the Meningitis example --
- S: Proposition that patient has stiff neck
- H: Proposition that patient has severe headache
- M: Proposition that patient has meningitis
- Meningitis causes stiff-neck, 50% of the time
- Meningitis causes head-ache, 70% of the time
- Probability of Meningitis should go up, if both symptoms reported -- how to combine such symptoms?

Combining Evidence

- $P(A | B \wedge C) = P(A \wedge B \wedge C) / P(B \wedge C)$
- Numerator:
 - $P(M \wedge S \wedge H) = P(S | M \wedge H) * P(M \wedge H)$
 - $= P(S | M) * P(M \wedge H) \rightarrow \text{WHY?}$
 - $= P(S | M) * P(H | M) * P(M)$

Conditional Independence

- Helps decompose larger tables into small ones
 - Allows probabilistic systems to scale up
- Conditional independence** much more common than absolute independence
- Decomposition of large probabilistic systems into weakly connected subsets through conditional independence
 - Considered one of the most important developments in the recent history of AI
- Will deal more in lectures on Belief nets

Decision Theory

(How to make decisions)

Decision Theory

= Probability theory + Utility Theory
(deals with chance) (deals with outcomes)

- Fundamental idea:**
 - The **MEU** (Maximum expected utility) principle
 - Agent is **rational** if and only if it chooses the action that yields the highest expected utility, averaged over all possible outcomes of the action
 - Weigh the utility of each outcome by the probability that it occurs

Application of MEU Principle

- $EU(\text{Plan1}) = P(\text{home-early} | \text{plan1}) * U(\text{home-early}) + P(\text{stuck1} | \text{plan1}) * U(\text{stuck1})$
 $= 0.8 * 100 + 0.2 * -1000 = -120$
- $EU(\text{Plan2}) = P(\text{home-somewhat-early} | \text{plan2}) * U(\text{home-somewhat-early}) + P(\text{stuck2} | \text{plan2}) * U(\text{stuck2})$
 $= 0.7 * 50 + 0.3 * -10 = 32$

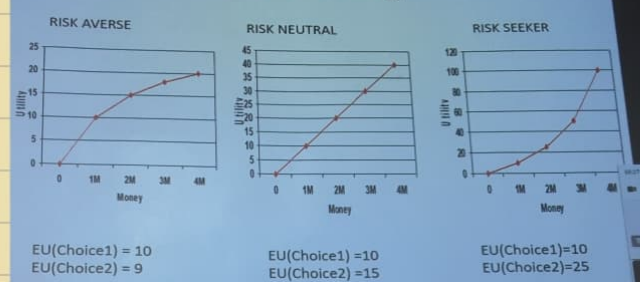
Lottery Example

- Suppose an agent gives you a choice:
 - Choice 1:** You will get \$1,000,000
 - Choice 2:** The agent will toss a coin
 - If heads, then you win \$3,000,000
 - If tails, then you get nothing
- Simple expected utility calculations give:
 - $EU(\text{Choice1}) = \$1,000,000$
 - $EU(\text{Choice2}) = \$1,500,000$
- So why did we prefer the first choice?

Risk Aversion

- We are **risk averse**
- Our utility functions for money are as follows (!!):
 - Our first million means a lot $U(\$1M) = 10$
 - Second million not so much $U(\$2M) = 15$ (NOT 20)
 - Third million even less so $U(\$3M) = 18$ (NOT 30)
 -

Risk Averse, Risk Neutral, Risk Seeking



More Risk Aversion

- Key: Slope of utility function is **continuously decreasing**
 - We will refuse to play a monetarily fair bet
- Suppose we start with x dollars
 - We are offered a game:
 - 0.5 chance to win 1000 dollars ($c = 1000$)
 - 0.5 chance to lose 1000 dollars ($c = 1000$)
 - Expected monetary gain or loss is zero (hence monetarily fair)
 - Should be neutral to it, but seems we are not! Why?
 - $U(x + c) - U(x) < U(x) - U(x - c)$
 - $U(x + c) + U(x - c) < 2 U(x)$
 - $[U(x + c) + U(x - c) / 2] < U(x)$
 - $EU(\text{playing the game}) < EU(\text{not playing the game})$

Multiattribute Utility Theory

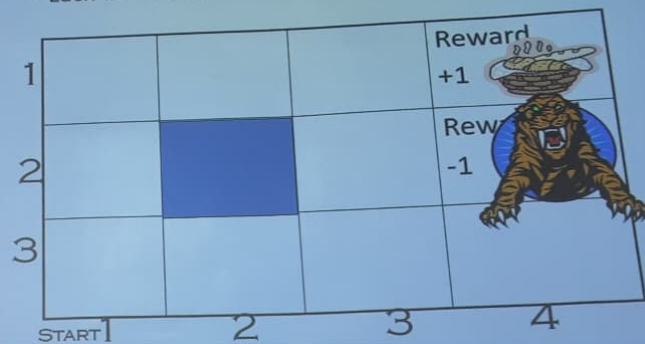
- Can we capture desirability of outcomes in a single utility function?
- Suppose renting an apartment
 - House1: closer-to-university, newer, costs 100 units
 - House2: Farther-from-university, older, costs 85 units
 - (Assume, you can afford up to 100 units)
- Outcomes characterized by two or more attributes
 - Attributes: $X_1, X_2, \dots, X_N$, e.g., <distance-to-univ, old/new, cost>
 - Values: $x_1, x_2, \dots, x_N$,
 - Closer-to-univ = 1, farther-from-univ = 0; new = 1, old = 0
 - Apartment1: <1,1,-100> Apartment2: <0,0,-85>
 - Which is a better apartment? (Pairwise comparison fails)

Markov Decision Process (MDP) Chapter 17: Making Complex Decisions

- Defined as a tuple: $\langle S, A, P, R \rangle$
 - **S**: State
 - **A**: Action
 - **P**: Transition function
 - Table $P(s' | s, a)$, prob of s' given action "a" in state "s"
 - **R**: Reward
 - $R(s, a)$ = cost or reward of taking action a in state s

Example: What SEQUENCE of actions should our agent take?

- Agent can take action N, E, S, W
- Each action costs $-1/25$



Evaluate Markov Chain

- Use **value iteration algorithm**, but with fixed action

• Initialize: $U_0(I) = 0$

• Iterate:

$$U_{t+1}(I) = R(I,A) + \sum_J P(J|I,A) * U_t(J)$$

- Until close-enough (U_{t+1}, U_t)

To compare two policies:

- Check the value of start state (1,3) given the two evaluations obtained above

Value Iteration: Modify

- Initialize: $U_0(I) = 0$

• Iterate:

$$U_{t+1}(I) = \max_A [R(I,A) + \gamma \sum_J P(J|I,A) * U_t(J)]$$

- Until close-enough (U_{t+1}, U_t)

Questions

- Now that we know how to evaluate a policy, what if we just search the entire space of policies one by one? Would that give us the optimal policy?
- How do you find best policy if agent will live for 3 steps?

Policy Iteration

- Start with some (random) policy
- Repeat until policy stabilizes (no change)
 - Determine value of policy π (see previous lecture)
 - Provides us a value of each state I given policy π
 - At each state I do:
 - If /* Check if policy can be improved at each state */

$$\max_A [R(I, A) + \sum_J P(J|I,A) * U_t(J) > R(I, \text{Policy}(I)) + \sum_J P(J|I, \text{Policy}(I)) * U_t(J)]$$

Same as Value iteration formula

Linear Programming

- Mathematical programming is used to find the **best or optimal solution** to a problem that requires a decision or set of decisions about how best to use a set of limited resources to achieve a state goal of objectives.
- Steps involved in mathematical programming
 - Conversion of stated problem into a mathematical model that abstracts all the essential elements of the problem.
 - Exploration of different solutions of the problem.
 - Finding out the most suitable or optimum solution.

The Linear Programming Model (1)

Let: $X_1, X_2, X_3, \dots, X_n$ = decision variables
 Z = Objective function or linear function

Requirement: Maximization of the linear function Z .

$$Z = c_1X_1 + c_2X_2 + c_3X_3 + \dots + c_nX_n \quad \dots \text{Eq (1)}$$

subject to the following constraints:

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2$$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_n$$

$$\text{all } x_j \geq 0$$

Linear Programming

- Mathematical programming is used to find the **best or optimal solution** to a problem that requires a decision or set of decisions about how best to use a set of limited resources to achieve a state goal of objectives.
- Steps involved in mathematical programming**
 - Conversion of stated problem into a mathematical model that abstracts all the essential elements of the problem.
 - Exploration of different solutions of the problem.
 - Finding out the most suitable or optimum solution.
- Linear programming** requires that all the mathematical functions in the model be **linear functions**.

The Linear Programming Model (1)

Let: $X_1, X_2, X_3, \dots, X_n$ = decision variables
 Z = Objective function or linear function

Requirement: Maximization of the linear function Z .

$$Z = c_1X_1 + c_2X_2 + c_3X_3 + \dots + c_nX_n \quad \dots \text{Eq (1)}$$

subject to the following **constraints**:

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2$$

$$\vdots$$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_n$$

$$\text{all } x_j \geq 0$$

The Linear Programming Model (2)

- The linear programming model can be written in more efficient notation as:

Maximize

$$Z = \sum_{j=1}^n c_j x_j$$

subject to:

$$\sum_{j=1}^n a_{ij} x_j \leq b_i$$

where

$$i = 1, 2, \dots, m$$

and

$$x_j \geq 0$$

where

$$j = 1, 2, \dots, n$$

The decision variables, $x_1, x_2, \dots, x_n$, represent levels of n competing activities.

Examples of LP Problems (1)

1. A Product Mix Problem

- A manufacturer has fixed amounts of different resources such as raw material, labor, and equipment.
- These resources can be combined to produce any one of several different products.
- The quantity of the i^{th} resource required to produce one unit of the j^{th} product is known.
- The decision maker wishes to produce the combination of products that will **maximize total income**.

Examples of LP Problems (2)

2. A Transportation Problem

- A product is to be shipped in the amounts $a_1, a_2, \dots, a_m$ from m shipping origins and received in amounts $b_1, b_2, \dots, b_n$ at each of n shipping destinations.
- The cost of shipping a unit from the i^{th} origin to the j^{th} destination is known for all combinations of origins and destinations.
- The problem is to determine the amount to be shipped from each origin to each destination such that the total **cost of transportation is a minimum**.

Examples of LP Problems (3)

3. A Flow Capacity Problem

- One or more commodities (e.g., traffic, water, information, cash, etc.) are flowing from one point to another through a network whose branches have various constraints and flow capacities.
- The direction of flow in each branch and the capacity of each branch are known.
- The problem is to determine the **maximum flow**, or capacity of the network.

Developing LP Model

XYZ Company produces two products: I and II. The raw material requirements, space needed for storage, production rates, and selling prices for these products are given in Table 1.

TABLE 1 Production Data for N. Dustrious Company

	Product	
	I	II
Storage space (ft ² /unit)	4	5
Raw material (lb/unit)	5	3
Production rate (units/hr)	60	30
Selling price (\$/unit)	13	11

The total amount of raw material available per day for both products is 1575 lb. The total storage space for all products is 1500 ft², and a maximum of 7 hours per day can be used for production.

Developing LP Model

- The object is to maximize the equation:

$$Z = 13x_1 + 11x_2$$

subject to the constraints on storage space, raw materials, and production time.

- Each unit of product I requires 4 ft² of storage space and each unit of product II requires 5 ft². Thus a total of $4x_1 + 5x_2$ ft² of storage space is needed each day. This space must be less than or equal to the available storage space, which is 1500 ft². Therefore,

$$4x_1 + 5x_2 \leq 1500$$

- Similarly, each unit of product I and II produced requires 5 and 3 lbs, respectively, of raw material. Hence a total of $5x_1 + 3x_2$ lb of raw material is used.

Developing LP Model

- This must be less than or equal to the total amount of raw material available, which is 1575 lb. Therefore,

$$5x_1 + 3x_2 \leq 1575$$
- Product I can be produced at the rate of 60 units per hour. Therefore, it must take 1/60 of an hour to produce 1 unit. Similarly, it requires 1/30 of an hour to produce 1 unit of product II. Hence a total of $x_1/60 + x_2/30$ hours is required for the daily production. This quantity must be less than or equal to the total production time available each day. Therefore,

$$x_1/60 + x_2/30 \leq 7 \text{ or } x_1 + 2x_2 \leq 420$$

Developing LP Model

- The linear programming model for this example can be summarized as:

Maximize

$$Z = 13x_1 + 11x_2$$

subject to:

$$4x_1 + 5x_2 \leq 1500$$

$$5x_1 + 3x_2 \leq 1575$$

$$x_1 + 2x_2 \leq 420$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

Linear Programming for MDPs

- Maximize

$$\sum_{i \in S} v_i \quad (S \text{ is the set of states, } v_i \text{ is value of state})$$

Such that:

$$v_i \leq [R(i,A) + \gamma \sum_j P(j|i,A) v_j]$$

Value Iteration Equation

Linear Programming

- Linear programming polynomial time (Karmarkar, 84)
 - Popular method Dantzig's simplex (1963)
 - Simplex performs well in practice, but could be exponential
- Although polynomial time typically slower than MDP specific algorithms such as value iteration
- Determine policy from V_i using method from previous slides
 - Yields a deterministic policy for MDPs: why?
- Key advantage: Many times it is amenable to modeling specific constraints

A more popular formulation

$$\max \sum_i \sum_a x_{ia} r_{ia}$$

$$\sum_a x_{ja} - \sum_i \sum_a x_{ia} p_{ij}^a = \alpha_j, \quad x_{ia} \geq 0$$

$x_{\{ia\}}$: Expected number of times action a is taken in state i
 $r_{\{ia\}}$: Reward for taking action a in state i
 $p_{\{a\{ij\}}$: Probability of reaching state j when action a is taken in state i
 $\alpha_{\{j\}}$: Initial probability of being in state j

Refactoring

$$\max \sum_i \sum_a x_{ia} r_{ia} \quad \left| \quad \sum_i \sum_a (\delta_{ij} - p_{ij}^a) x_{ia} = \alpha_j, \quad x_{ia} \geq 0, \right.$$

where δ_{ij} is the Kronecker delta, defined as $\delta_{ij} = 1 \iff i = j$.

MDP Example

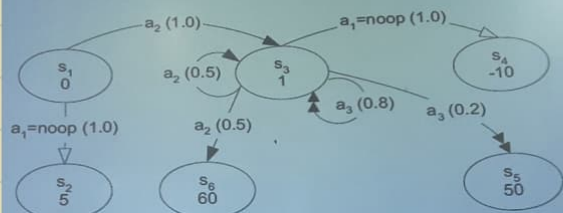
$$\max(rx) \mid Ax = \alpha, x \geq 0,$$

$$x = [(x_{11}, x_{12}), x_{21}, (x_{31}, x_{32}, x_{33}), x_{41}, x_{51}, x_{61}]^T,$$

$$r = [(0, 0), 5, (1, 1, 1), -10, 50, 60], \quad \alpha = [0.1, 0.1, 0.1, 0.1, 0.1, 0.5]^T,$$

$$A = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0.5 & 0.2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.2 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -0.5 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

MDP Example



A simple MDP: $S = \{s_1, \dots, s_6\}, A = \{a_1 = \text{noop}, a_2, a_3\}$

Solution to the LP

$$x = [(0, 0.1), 0.1, (0, 0.4, 0), 0.1, 0.1, 0.7]$$

$$d = [a_2, a_1, a_2, a_1, a_1, a_1]$$

Solving the LP we get x , which maps to a deterministic uniformly-optimal policy

We get the following output if we change alpha to the following

$$\alpha = [1, 0, 0, 0, 0, 0]$$

$$x = [(0, 1), 0, (0, 2, 0), 0, 0, 1]$$

$$d_1 = a_2, d_3 = a_2, d_6 = a_1$$

arbitrary actions for s_2, s_4 , and s_5

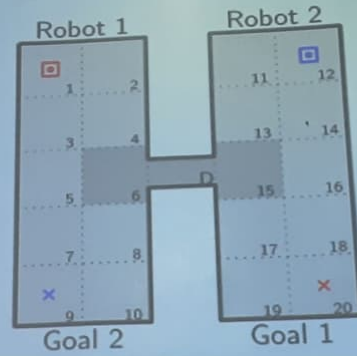
CMDP

- A key advantage of the formulation is the ability to model constraints
- If we want to say state 1 must have all actions take with equal probability
 - $x_{11} = x_{12}$
- Addition of such constraints results in a CMDP (Constrained MDP)
- How do you model action 1 in state 1 must be taken 30% of time?
 - $x_{11}/(x_{11}+x_{12}) = 1/3$
 - How can you do this with value or policy iteration?

Dec-MDP

- In MMDP model agent observes the joint state
- Many times in a joint problem, an agent may observe only his/her local state
- Separating out the policy computation for each agent is not an option since they have joint transitions and rewards
- Dec-MDP – A formal framework to address these issues

Robots coordinating in a hallway with a narrow passage



Can be modeled using Dec-MDP framework